# Do Now · retrieval
## Floating-point addition

Name: _____ Class: _____ Date: _____

Answer four retrieval questions before reviewing earlier learning.

### Q1

Convert 01011000 0011 to denary. Use an eight-bit two’s-complement mantissa and a four-bit two’s-complement exponent.

### Q2

Represent 0.375 in normalised floating-point form using an eight-bit two’s-complement mantissa and a four-bit two’s-complement exponent.

### Q3

Convert 11110110 from 8-bit two’s-complement binary to denary. Show signed place values.

### Q4

Convert $AD_{16}$ to binary, then logically shift the 8-bit value right once. Give the final denary value.

# New Learning · key ideas

The important knowledge from today's lesson. Read, annotate, and explain it.

## Core idea 1

Align the exponents — Choose the larger exponent. Increase the smaller exponent one step at a time and right-shift that mantissa once for each step.

## Core idea 2

Preserve sign and working precision — An arithmetic right shift repeats 0 for a positive mantissa and 1 for a negative mantissa. — Keep low-order guard bits when a shift would otherwise discard a non-zero bit.

## Core idea 3

Add using a wider working sum — Sign-extend both aligned mantissas before adding so the calculation cannot wrap into the wrong sign. — CODE:
requested mantissa: 6 bits
working sum: 7 bits

## Annotate the key visual

Explain which labels or features show the main process.

# New Learning · applying the ideas

## Core idea 4

Normalise and check — If the sum is too large for the mantissa, shift right and increase the exponent. If redundant sign bits remain, shift left and decrease the exponent.

## Vocabulary · use the correct technical terms

Exponent alignment — Rewriting operands with a common exponent before adding mantissas.

Arithmetic right shift — A right shift that repeats the sign bit.

Sign extension — Repeating a sign bit when widening a signed working value.

Guard bits — Extra low-order working bits that reduce premature loss of precision.

## Independent skill check · apply the lesson

Add -5 and +3 with a 6-bit two's-complement mantissa and 4-bit exponent. Show alignment, signed addition and normalisation.

More animated examples are available in the original online lesson.

# I Do · worked example

## Problem

Add 5 + 1.5

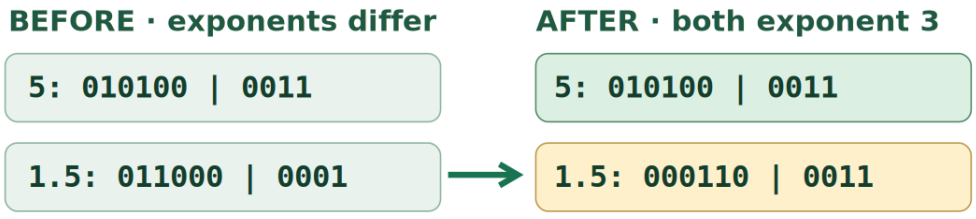
## Follow the worked method

1. Use the common exponent 3 — Raise the second exponent from 1 to 3, so shift its mantissa right twice using sign extension. — CODE:
010100 0011
011000 0001 → 000110 0011

2. Return to the requested width — CODE:
0011010
→ 011010 0011 — The mantissa already begins 01 , so the exponent stays at 3.

3. Verify in denary — CODE:
5 + 1.5 = 6.5

**Source model · final illustrated step**




**Arithmetic right shift ×2 · raise exponent 1 → 3**

Mantissa | exponent · each number keeps its value

## Why this method works

Identify the decision, rule or invariant that makes the method correct.

# We Do · Try → Discuss → Improve

TRY independently in BLACK PEN; compare methods with a partner; improve your reasoning.

## Guided problem · TRY

Build the answer

## DISCUSS

Compare how you reached the answer. Where could someone make an error?

## IMPROVE

Add one precise correction or a stronger reason. Answers are not printed here.

# You Do · independent practice

Answer these selected original questions in BLACK PEN. Other exam tasks remain online.

## Question 1

Question 1 · OCR OCTOBER 2021 Q2(c) · [5]

Add 011000 0110 and 010100 0100 . Show your working and give your answer in normalised floating-point form using a six-bit two’s-complement mantissa and a four-bit two’s-complement exponent.

Paraphrased exam question; original operands and marks retained. OCR October 2021, Q2(c) .

Check that you have used the technical terms and explained your decisions.

# You Do · extended practice

Answer these selected original questions in BLACK PEN. Other exam tasks remain online.

## Question 2

Question 4 · ORIGINAL EXAM-STYLE EXPLANATIONS · [5]
Why must exponents match before mantissas are added directly? [2]
What enters from the left when a negative mantissa is shifted right for alignment, and why? [2]
Why can addition lose precision even if the result is within exponent range? [1]

Check that you have used the technical terms and explained your decisions.

# Exit Ticket · reflect and retrieve

## Exit Ticket

Floating-point mantissas must be aligned to the same exponent before they are added directly.

## Use key vocabulary from memory

1. Explain Exponent alignment in your own words.

2. Explain Arithmetic right shift in your own words.

3. Explain Sign extension in your own words.

## Confidence and next step

How confident are you? Thumbs up / side / down

Which question would you improve after checking the classroom model?

Check answers after the lesson using the teacher-guided source models.